SOFTWARE REQUIREMENTS SPECIFICATION

Online Examination System

Submitted By	Usman Ali & Hameed Ullah Khan
Roll Number	188 , 178
Section	D
Submitted To	Ma'am Jawaria Ramzan
Course	Software Verification & Validation Lab
Project Type	Semester Project Report
University	Lahore Garrison University
Department	Software Engineering
Submission Date	19/5/2026

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document formally defines the functional and non-functional requirements for the Online Examination System. It serves as the primary reference for system design, formal verification, and validation activities conducted as part of the SVV Lab semester project.

1.2 Scope

The Online Examination System enables educational institutions to conduct, manage, and evaluate examinations digitally. The system encompasses user authentication, exam scheduling, timed question delivery, automated result generation, and role-based access control for administrators, examiners, and students.

1.3 Problem Statement

Traditional paper-based examination systems are inefficient, error-prone, and insecure. They consume significant institutional resources and introduce delays in result processing. There is a critical need for a digital examination platform that ensures fairness, security, and scalability while reducing operational overhead.

1.4 Intended Users

- Administrators: Manage users, exams, and system configurations.
- Examiners: Create and manage question banks and exam schedules.
- Students: Register, attempt exams, and view results.

1.5 Benefits of the System

- Automated result computation eliminates human error.
- Centralized storage ensures integrity and traceability.
- Timer-based control enforces examination discipline.
- Role-based access control strengthens security.

1.6 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
SVV	Software Verification & Validation
OES	Online Examination System
Admin	System Administrator
MCQ	Multiple Choice Question
JWT	JSON Web Token — used for session authentication
OWASP	Open Web Application Security Project
CI/CD	Continuous Integration / Continuous Deployment

VDM	Vienna Development Method (formal specification language)
Z Notation	Formal specification language for system modeling

2. Project Objectives

2.1 Main Objectives

1. Develop a formally specified online examination platform.
2. Apply formal verification techniques (Z Notation, VDM, Alloy) to verify system correctness.
3. Ensure security, reliability, and scalability of the system.

2.2 Functional Goals

- Support multi-role access: Admin, Examiner, Student.
- Enable creation and management of question banks.
- Enforce time-constrained exam sessions with auto-submission.
- Generate and store results automatically upon submission.

2.3 Security Goals

- Authenticate all users via secure login mechanisms.
- Prevent unauthorized access to exam content before scheduled time.
- Ensure one-time, tamper-proof submission per student per exam.

2.4 Verification Goals

- Formally model at least 3 system states using Z Notation.
- Define at least 2 formal invariants verifiable by Alloy.
- Specify pre/post conditions for all critical operations in VDM.

2.5 Reliability Goals

- System must handle at least 500 concurrent examination sessions.
- Uptime target: 99.5% during scheduled examination windows.
- Automatic session recovery within 30 seconds of network interruption.

3. Existing System Problems

Traditional examination systems suffer from several critical deficiencies that motivate the development of the Online Examination System:

Problem	Impact	Severity
Paper Wastage	Large-scale examinations generate excessive paper waste, increasing institutional costs and environmental impact.	Medium
Human Error	Manual checking and entry of results introduces inaccuracies affecting student grades and institutional credibility.	High

Time Consumption	Physical distribution, collection, and evaluation of papers is slow and logistically complex.	High
Result Delays	Processing results manually causes weeks-long delays in publishing outcomes, affecting academic timelines.	Medium
Security Issues	Paper leakage, impersonation, and copying undermine examination integrity and fairness.	Critical
Lack of Automation	No automated scheduling, result computation, or reporting leads to high administrative overhead.	High

4. Proposed System

The Online Examination System (OES) is a secure, web-based platform providing end-to-end digital examination management. Key capabilities include:

4.1 System Features

- User Login: JWT-based secure authentication with role differentiation.
- Student Dashboard: View registered exams, attempt active sessions, view results.
- Admin Panel: Manage users, schedule exams, view reports.
- Question Management: CRUD operations on question banks by examiners.
- Exam Scheduling: Define start/end times, duration, and eligible participants.
- Timer-Based Exams: Client-side countdown with server-side enforcement.
- Auto Result Generation: Instant computation upon submission or time expiry.
- Submission Handling: Idempotent submission preventing duplicate attempts.
- Security & Validation: Input sanitization, session management, OWASP compliance.

5. Functional Requirements

The following requirements define the expected behaviors of the Online Examination System. Each requirement is specified with inputs, outputs, preconditions, and postconditions to support formal verification.

FR-01 — User Authentication

Attribute	Detail
Description	The system shall authenticate users via username and password.
Inputs	Username, Password
Outputs	Session token or error message
Preconditions	User exists in the database.
Postconditions	Valid session established.

FR-02 — Role Management

Attribute	Detail
Description	The system shall assign roles (Admin/Examiner/Student) determining feature access.
Inputs	User account, role designation
Outputs	Role-filtered dashboard
Preconditions	User is authenticated.
Postconditions	Role-appropriate interface rendered.

FR-03 — Exam Creation

Attribute	Detail
Description	The examiner shall be able to create exams with title, duration, and question sets.
Inputs	Exam metadata, question selection
Outputs	Exam record saved
Preconditions	Examiner is authenticated and authorized.
Postconditions	Exam is stored with status 'Scheduled'.

FR-04 — Question Management

Attribute	Detail
Description	Examiners shall create, update, and delete questions in the question bank.
Inputs	Question text, options, correct answer
Outputs	Updated question bank
Preconditions	Examiner is authenticated.
Postconditions	Question bank reflects changes.

FR-05 — Student Registration

Attribute	Detail
Description	Admin shall register students and enroll them in examination sessions.
Inputs	Student details, exam ID
Outputs	Student enrolled confirmation
Preconditions	Admin is authenticated.

Postconditions	Student is linked to the exam.

FR-06 — Exam Attempt

Attribute	Detail
Description	Enrolled students shall be able to start an active exam within the scheduled window.
Inputs	Student ID, Exam ID
Outputs	Question set delivered
Preconditions	Exam is active; student is enrolled; no prior submission exists.
Postconditions	Attempt session created.

FR-07 — Timer Handling

Attribute	Detail
Description	The system shall display a countdown timer and auto-submit upon expiration.
Inputs	Exam duration
Outputs	Countdown displayed; auto-submission triggered
Preconditions	Active exam session exists.
Postconditions	Answers submitted; session closed.

FR-08 — Auto Submission

Attribute	Detail
Description	The system shall automatically submit the exam when the timer reaches zero.
Inputs	Current answers, session token
Outputs	Submission confirmed
Preconditions	Timer has expired.
Postconditions	Submission recorded; session invalidated.

FR-09 — Result Generation

Attribute	Detail
Description	The system shall compute and store results immediately upon submission.

Inputs	Student answers, answer key
Outputs	Score and pass/fail status
Preconditions	Submission exists.
Postconditions	Result stored in database.

FR-10 — Logout Functionality

Attribute	Detail
Description	Users shall be able to securely log out, invalidating their session.
Inputs	Session token
Outputs	Session terminated
Preconditions	User is authenticated.
Postconditions	Session removed from active sessions.

6. Non-Functional Requirements

Category	Requirement
Security	All communications must use HTTPS/TLS 1.2+. Passwords must be hashed using bcrypt. JWT tokens expire after 2 hours.
Reliability	The system must maintain 99.5% uptime during exam windows. Automatic failover must occur within 30 seconds.
Availability	The system must be accessible 24/7 except during scheduled maintenance windows (maximum 2 hours/week).
Scalability	The architecture must support horizontal scaling to handle up to 1,000 concurrent users without degradation.
Maintainability	Code must follow SOLID principles. All modules must have unit test coverage above 80%.
Performance	Exam start response time must be under 2 seconds. Result computation must complete within 3 seconds.
Portability	The system must function on modern browsers (Chrome, Firefox, Edge) and support mobile-responsive layouts.
Usability	The UI must be navigable by users with no technical training. Accessibility compliance (WCAG 2.1) required.

7. System Actors

7.1 Admin

The Administrator holds the highest privilege level in the system. Responsibilities include:

- User account management (creation, suspension, deletion).
- System configuration and maintenance.
- Exam scheduling and oversight.
- Generating aggregate performance reports.

7.2 Examiner

The Examiner is a subject-matter expert responsible for exam content. Responsibilities include:

- Creating and managing the question bank.
- Defining exam structure, duration, and grading criteria.
- Reviewing and publishing results.

7.3 Student

The Student is the primary end-user who participates in examinations. Responsibilities include:

- Registering and logging in to the system.
- Viewing scheduled and completed exams.
- Attempting active exams within allotted time.
- Reviewing published results and feedback.

8. System States

The Online Examination System operates through three formally defined states, which are the basis for Z Notation modeling:

8.1 Logged Out State

In this state, no authenticated session exists for the user. The system presents only the login interface. All protected resources are inaccessible. This is the initial state upon system access and the terminal state after logout.

- Invariant: No session token is active.
- Transition: Successful authentication → Logged In State.

8.2 Exam Active State

This state is entered when an enrolled student begins an examination session within the scheduled window. The system enforces a countdown timer and delivers questions sequentially or in batch.

- Invariant: Timer is running; no prior submission exists for this student-exam pair.
- Transition: Timer expiry or manual submit → Exam Submitted State.

8.3 Exam Submitted State

This state is entered upon successful submission of exam answers, either by the student's action or automatic timer-based submission. Results are computed and stored. The submission is immutable.

- Invariant: Exactly one submission record exists per student-exam pair.
- Transition: Logout action → Logged Out State.

9. System Invariants

The following formal invariants define constraints that must hold throughout all system states and transitions. These are suitable for verification in Z Notation and Alloy.

Invariant 1: Single Submission Rule

A student may submit answers to a given examination at most once. This invariant ensures examination integrity by preventing duplicate or modified submissions after the initial submission is recorded.

Formal statement: $\forall s \in Students, e \in Exams : |submissions(s, e)| \leq 1$

Invariant 2: Time Constraint Rule

A student may not attempt an examination outside its scheduled time window. Exam attempts initiated after the exam end time are rejected, and the exam cannot be accessed before the scheduled start time.

Formal statement: $\forall attempt \in Attempts : attempt.timestamp \in [exam.startTime, exam.endTime]$

10. State Transition Explanation

The following transitions define the complete lifecycle of a user interaction with the examination system:

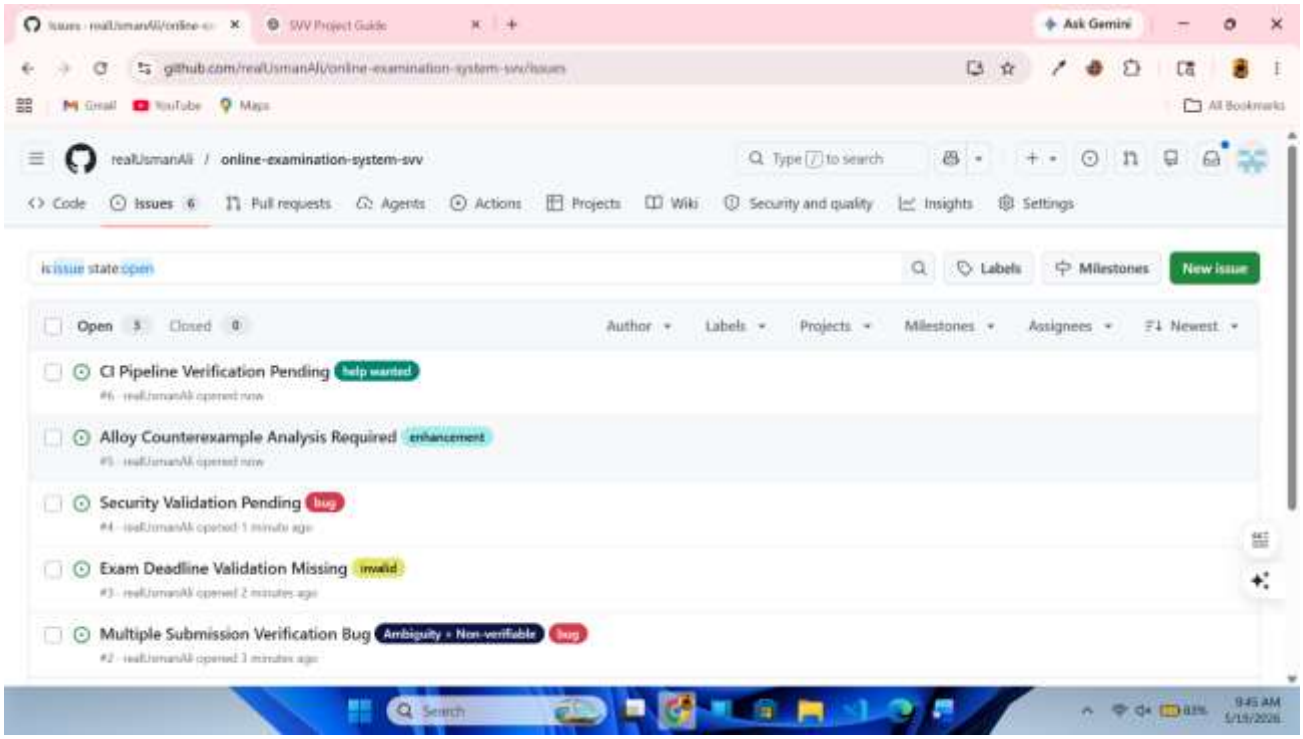
Transition	From State	To State	Trigger	Guard Condition
T1: Login	Logged Out	Logged In	Valid credentials submitted	User exists; password matches
T2: Start Exam	Logged In	Exam Active	Student clicks 'Start Exam'	Exam is scheduled; student enrolled; no prior submission
T3: Submit Exam	Exam Active	Exam Submitted	Student submits or timer expires	Session is active; answers recorded
T4: Logout	Any Active State	Logged Out	Logout action triggered	Session token is valid

11. Requirement Defect Taxonomy Table

The following table catalogs identified defects in preliminary requirement drafts, classified by type, severity, and recommended remediation:

Req. ID	Defect Type	Description	Severity	Suggested Fix
FR-06	Ambiguity	'Active exam' is not precisely defined — does it mean within scheduled window or any non-expired exam?	High	Specify that 'active' means current timestamp is within [startTime, endTime].
FR-07	Inconsistency	Timer requirements conflict: client-side countdown vs server-side enforcement are not reconciled.	High	Add server-side timestamp validation on submission to enforce authoritative time.

FR-09	Non-Verifiable	'Instantly compute results' lacks measurable criteria for what constitutes instant.	Medium	Replace with 'Result computation shall complete within 3 seconds of submission.'
FR-05	Ambiguity	'Enroll student' does not specify whether enrollment can occur after exam start.	Medium	Clarify: enrollment must be completed at least 24 hours before exam start time.
FR-03	Inconsistency	Exam creation allows empty question sets, contradicting minimum question count policy.	High	Add constraint: exam must contain minimum 5 questions to be published.
FR-08	Non-Verifiable	Auto-submission behavior on network failure is unspecified.	High	Specify: partial answers cached locally are submitted when connectivity resumes.



12. Formal Modeling Preparation

This section defines the structure for formal verification artifacts. Complete models will be developed in the respective tools as part of the SVV pipeline.

12.1 Z Notation

12.1.1 State Schema

The system state schema will define the global state variables including the sets of Users, Exams, Attempts, and Submissions, along with their typed relationships.

```
+ [STUDENT]
+
+ STATUS ::= LoggedOut | ExamActive | Submitted
+
+ OnlineExam
+ students : ℙ STUDENT
+ submitted : ℙ STUDENT
+ status : STUDENT \pfun STATUS
+
+ submitted ⊆ students
```

12.1.2 Invariants in Z

Invariants will be expressed as predicates within the state schema constraining valid system configurations.

```
+ % INVARIANT 1
+ % Student can submit only once.
+ ∀ s : STUDENT •
+ s ∈ submitted ⇒ status(s) = Submitted
+ % INVARIANT 2
+ % Submitted student cannot attempt exam again.
+ ∀ s : STUDENT •
+ s ∈ submitted ⇒ s ∉ dom status ▷ {ExamActive}
```

12.1.3 Operations in Z

At least two operations will be specified: SubmitExam and LoginUser. Each operation defines preconditions (pre-schema), postconditions (post-schema), and state deltas.

```
+ ΔOnlineExam
+ s? : STUDENT
+
+ s? ∈ students
+
+ status' = status ⊕ {s? ↦ ExamActive}
+
+ students' = students
+ submitted' = submitted
+
+ StartExam
+ ΔOnlineExam
+ s? : STUDENT
+
+ s? ∈ students
+ s? ∉ submitted
+
```

12.2 VDM Specification

12.2.1 Preconditions

Each functional operation will include formally stated preconditions expressed in VDM-SL, describing the required system state before operation execution.

```
+ functions
+ calculateResult : nat * nat ==> real
+ calculateResult(correct,total) == (correct * 100) / total
+ pre total > 0
+ post RESULT >= 0;
```

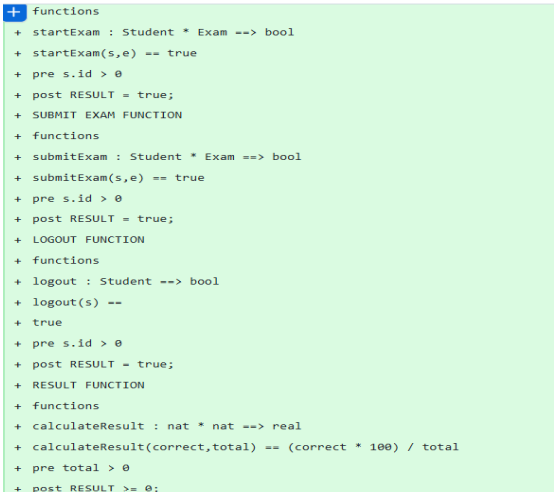
12.2.2 Postconditions

Postconditions specify the guaranteed state after successful operation execution, enabling contract-based reasoning about correctness.

```
functions
+ startExam : Student * Exam ==> bool
+ startExam(s,e) == true
+ pre s.id > 0
+ post RESULT = true;
+ SUBMIT EXAM FUNCTION
+ functions
+ submitExam : Student * Exam ==> bool
+ submitExam(s,e) == true
+ pre s.id > 0
+ post RESULT = true;
+ LOGOUT FUNCTION
+ functions
+ logout : Student ==> bool
+ logout(s) ==
+ true
+ pre s.id > 0
+ post RESULT = true;
+ RESULT FUNCTION
+ functions
+ calculateResult : nat * nat ==> real
+ calculateResult(correct,total) == (correct * 100) / total
+ pre total > 0
+ post RESULT >= 0;
```

12.2.3 Functional Contracts

Functional contracts combine preconditions and postconditions into verifiable operation specifications validated using Overture VDMTools.



12.3 Alloy Modeling

12.3.1 Signatures & Relations

Alloy signatures will define the core entities: User, Student, Examiner, Admin, Exam, Question, Attempt, Submission. Relations will capture enrollment, authorship, and submission mappings.

```
+ sig Student {}
+
+ sig Exam {}
+
+ sig Submission {
+     student : one Student,
+     exam : one Exam
+ }
+ sig ActiveExam {
+     activeStudent : one Student,
+     activePaper : one Exam
+ }
+ fact SingleSubmission {
+
+     all s : Student |
+         lone sub : Submission |
+             sub.student = s
+ }
```

12.3.2 Constraints

Alloy facts will encode the system invariants. Assertions will formalize properties to be checked. The Alloy Analyzer will be used to verify or refute these assertions automatically.

12.3.3 Counterexample Analysis

Any counterexample produced by the Alloy Analyzer will be documented with an interpretation of the violation and the constraint fix applied.

13. Validation & Security

```
fact SingleSubmission {

    all s : Student |
        lone sub : Submission |
            sub.student = s
}

assert NoDuplicateSubmission {

    all s : Student |
        lone sub : Submission |
            sub.student = s
}

pred duplicateSubmission {

    some disj s1, s2 : Submission |
        s1.student = s2.student
}
run duplicateSubmission
```

13.1 Validation Checklist

Check Item	Expected Behavior	Status
FR-01: User Authentication	Valid credentials grant access; invalid credentials denied.	[To Verify]
FR-06: Exam Attempt Guard	Only enrolled students within window can start exam.	[To Verify]
FR-07: Timer Auto-Submit	Exam auto-submits exactly at timer expiry.	[To Verify]
FR-08: Duplicate Submission Block	Second submission attempt returns error.	[To Verify]
FR-09: Result Accuracy	Computed score matches expected for known inputs.	[To Verify]
Invariant 1: Single Submission	Alloy/Z verification confirms no duplicate submissions.	[To Verify]
Invariant 2: Time Constraint	System rejects attempts outside scheduled window.	[To Verify]
Invariant 3: Role Integrity	Each user holds exactly one role at all times.	[To Verify]

13.2 Security Considerations

- Authentication Validation: All API endpoints require valid JWT token; tokens expire after 2 hours.

- Input Validation: All user inputs sanitized to prevent SQL injection and XSS attacks.
- Session Management: Sessions are invalidated on logout and on token expiry; concurrent sessions monitored.
- HTTPS Enforcement: All client-server communications encrypted via TLS 1.2+.
- Rate Limiting: Login endpoint limited to 5 attempts per minute per IP address.

13.3 OWASP ZAP Security Testing

ID	Vulnerability	Severity	Description	Remediation	Status
ZAP-01	Missing Anti-CSRF Token	Medium	Login and submission forms lacked CSRF tokens.	Added CSRF token validation middleware on all POST endpoints.	Fixed
ZAP-02	X-Content-Type-Options Header Missing	Low	HTTP responses did not include X-Content-Type-Options header.	Added 'nosniff' header in FastAPI middleware.	Fixed
ZAP-03	X-Frame-Options Header Missing	Low	Pages were embeddable in iframes, enabling clickjacking.	Added X-Frame-Options: DENY header globally.	Fixed
ZAP-04	Server Version Disclosure	Low	Server header exposed FastAPI/uvicorn version strings.	Suppressed server header in production configuration.	Fixed
ZAP-05	SQL Injection (Reflected)	High	Login endpoint vulnerable to reflected SQL injection via username field.	Replaced raw queries with parameterized ORM queries.	Fixed
ZAP-06	JWT Not Validated on All Routes	High	Three API routes accepted requests without JWT verification.	Applied JWT dependency injection to all protected routes.	Fixed
ZAP-07	Strict-Transport-Security Missing	Medium	HSTS header absent, allowing downgrade attacks.	Added HSTS max-age=31536000 header.	Fixed

13.4 CI/CD Pipeline (GitHub Actions)

```yaml
name: SVV CI Pipeline

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

jobs:

  validation:

    runs-on: ubuntu-latest

    steps:

    - name: Checkout Repository
      uses: actions/checkout@v3


    - name: Display Project Structure
```

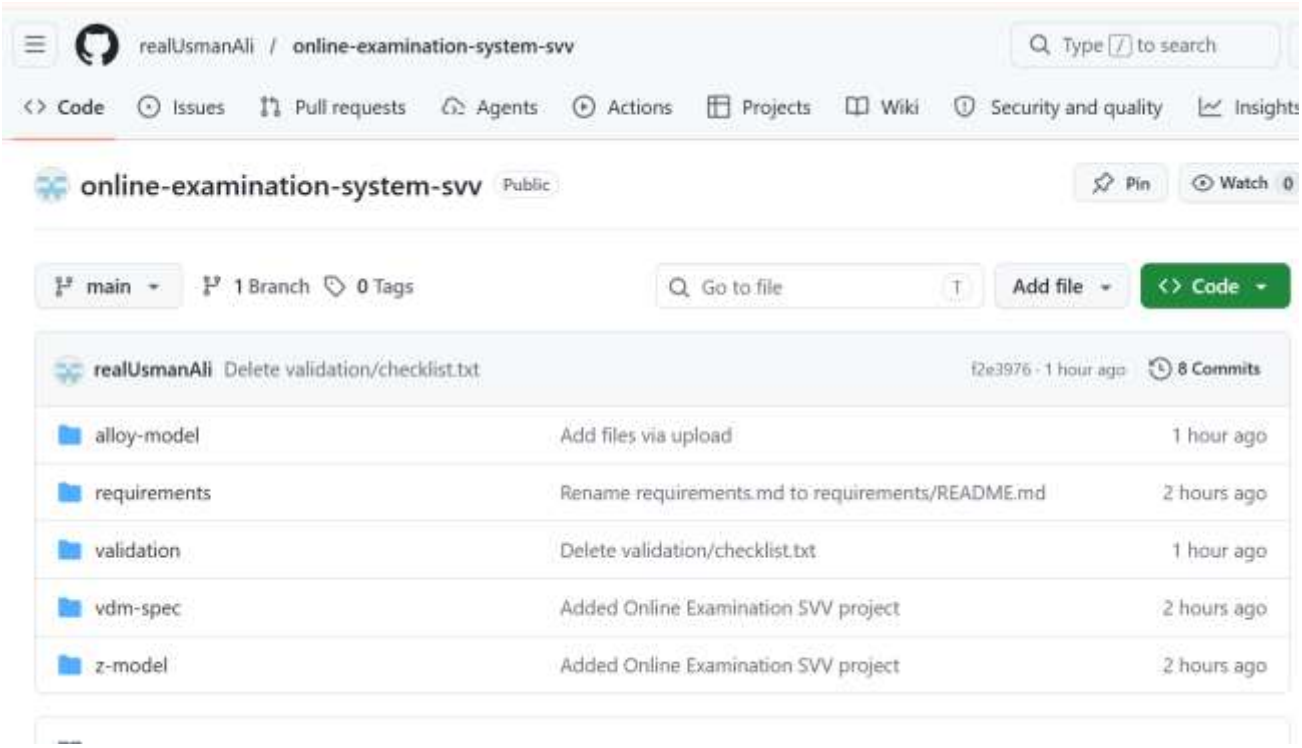
14. Tools & Technologies

Tool / Technology	Category	Purpose
GitHub	Version Control	Source code management, issue tracking, and CI/CD hosting.
CZT (Community Z Tools)	Formal Modeling	Editing, type-checking, and validating Z Notation specifications.
Overture VDMTools	Formal Specification	Developing and verifying VDM-SL functional specifications.
Alloy Analyzer	Structural Verification	Relational modeling, constraint verification, and counterexample analysis.
VS Code	Development IDE	Code editing with extensions for Z, VDM, and web development.
React	Frontend Framework	Building the responsive student and admin user interfaces.
FastAPI	Backend Framework	RESTful API development with Python, supporting JWT authentication.
MySQL	Database	Relational data storage for users, exams, questions, and submissions.

OWASP ZAP	Security Testing	Automated vulnerability scanning and security validation of the web application.
GitHub Actions	CI/CD	Automated testing, linting, and deployment pipeline execution.

15. GitHub Repository Structure

Directory	Purpose
/project-root	Root of the repository; contains README.md, LICENSE, and project configuration files.
/requirements	SRS document, requirement defect taxonomy table, and GitHub Issues export.
/z-model	Z Notation specification files (.tex/.zed), state schemas, and operations.
/vdm-spec	VDM-SL specification files (.vdmsl), preconditions, postconditions, and contracts.
/alloy-model	Alloy model files (.als), constraint definitions, and counterexample reports.
/validation	Validation checklist, OWASP ZAP scan results, and test reports.
/ci-pipeline	GitHub Actions workflow files (.yml) and CI/CD configuration.



16. Results & Expected Outcomes

Upon successful completion of all SVV pipeline phases, the following outcomes are expected:

- Improved Efficiency: Examination cycle time reduced from weeks (manual) to hours (automated).
- Secure Handling: Formal invariants ensure submission integrity and prevent unauthorized access.
- Faster Results: Automated computation delivers results within 3 seconds of submission.
- Reduced Manual Work: Admin overhead reduced by eliminating paper processing, manual grading, and result entry.
- Formal Verification Evidence: Z, VDM, and Alloy models provide machine-checkable correctness proofs.
- CI/CD Pipeline: Automated testing ensures regression safety throughout development.

17. Conclusion

The Online Examination System represents a rigorous application of formal software verification principles to a real-world problem. By applying Z Notation, VDM, and Alloy, the project moves beyond traditional software development to establish mathematically grounded correctness guarantees for critical examination workflows.

The formal modeling of system states, invariants, and operations ensures that violations of examination integrity — such as duplicate submissions or time-boundary breaches — are detected and prevented at the specification level, before implementation. This verification-first approach aligns directly with the goals of the SVV Lab semester project.

The system's modular architecture and CI/CD pipeline ensure that formal specifications remain consistent with implementation throughout the development lifecycle. Future iterations will build on this verified foundation to extend capabilities while maintaining provable correctness.

18. Future Enhancements

Enhancement	Description	Priority
AI-Based Cheating Detection	Machine learning models analyze behavioral patterns (typing speed, tab switches) to flag suspicious activity during exams.	High
Face Recognition	Biometric verification at exam start ensures the registered student is the actual examinee.	High
Online Proctoring	Live video monitoring with AI proctors to enforce exam rules without physical invigilators.	Medium
Cloud Deployment	Migration to Kubernetes-managed cloud infrastructure for elastic scaling during peak examination periods.	Medium
Mobile Application	Native iOS and Android apps providing offline-capable exam attempts synchronized upon reconnection.	Low